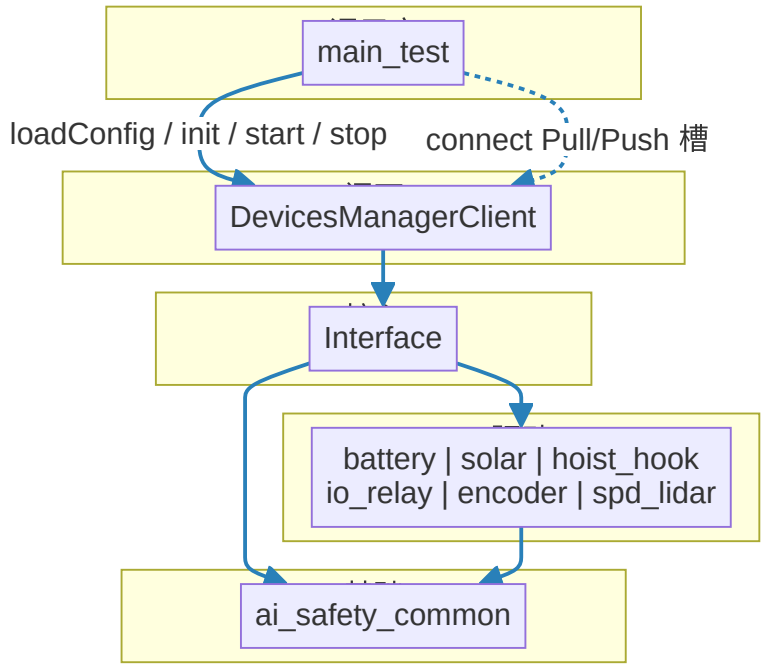
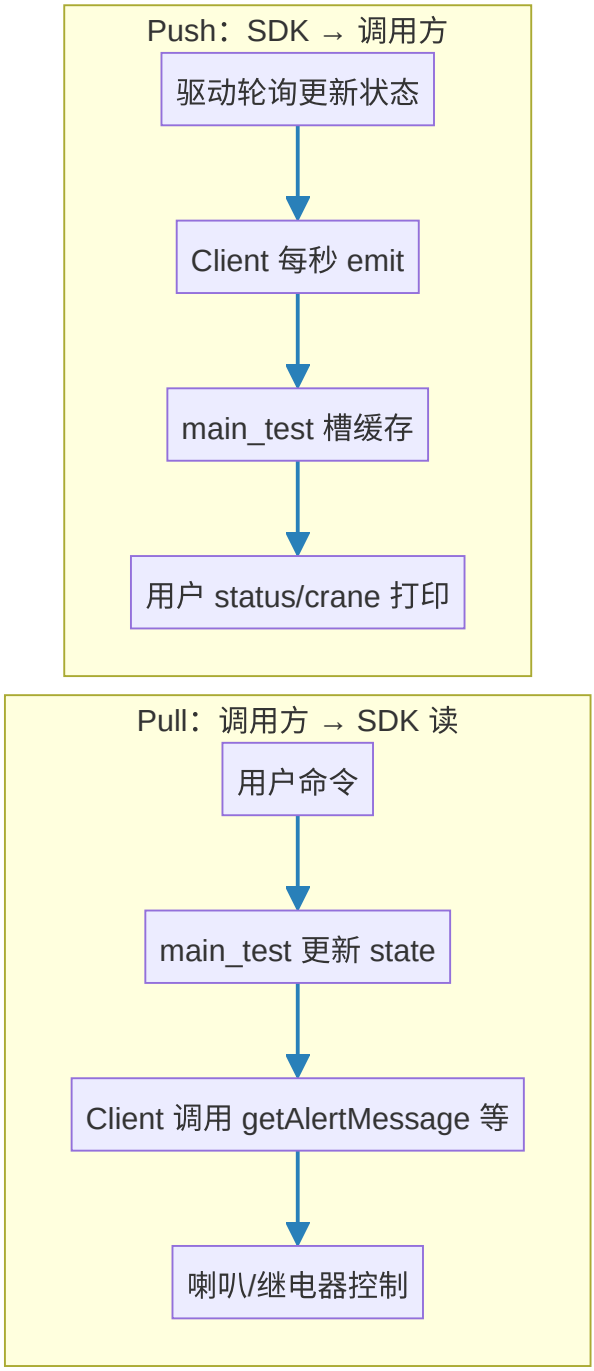
以 main_test 为调用节点的 SDK 逻辑图

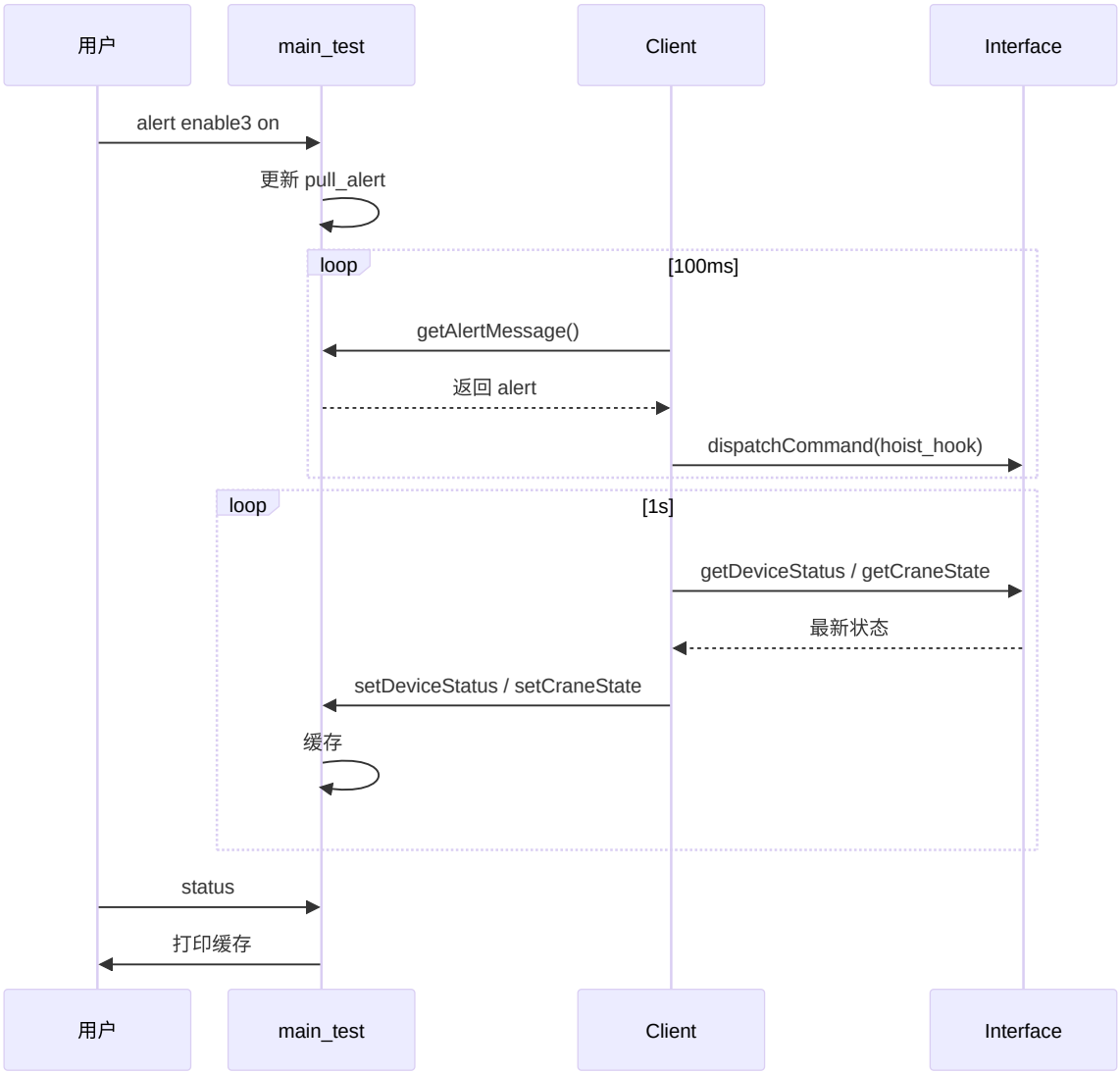
1. 总体架构



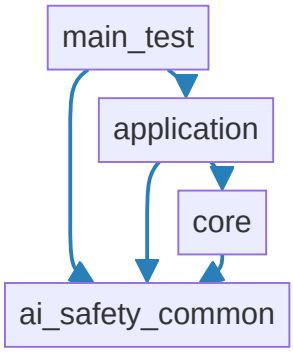
2. 数据流（Pull / Push）


Push：SDK → 调用方
驱动轮询更新状态
Client 每秒 emit
main_test 槽缓存
用户 status/crane 打印
Pull：调用方 → SDK 读
用户命令
main_test 更新 state
Client 调用 getAlertMessage 等
喇叭/继电器控制

3. Slot 交互时序



4. 模块依赖



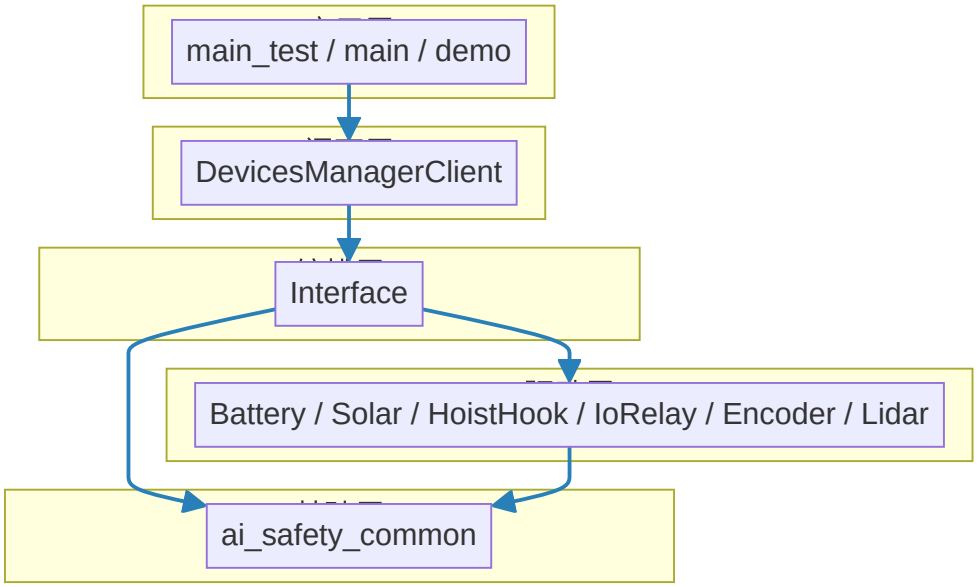
5. 简要说明

节点	说明
main_test	以 DevicesManagerClient 为唯一 SDK 入口的测试程序；通过终端命令驱动 Pull 槽、查看 Push 槽缓存。

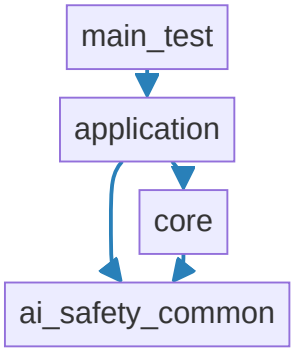
DevicesManagerClient	门面类，持有 Interface；暴露 loadConfig/init/start/stop、getDeviceStatus/getCraneState、以及 4 个 boost::signals2 槽。
Pull 槽	getAlertMessage、getBatteryButtonSignals：由调用方 connect 提供实现，Client 内部定时调用并用于喇叭控制、继电器控制。
Push 槽	setDeviceStatus、setCraneState：由 Client 内部每秒从 Interface 取最新状态并 emit，调用方 connect 接收。
Interface	加载配置、管理各 DriverAdapter 与 Core 驱动实例、维护 latest_device_status_/latest_crane_state_/latest_alert_message_ 等，供 Client 与自动轮询更新。
DriverAdapter	统一接口 init/start/stop/query；Interface 内通过 FunctionDriverAdapter 绑定到 battery/solar/hoist_hook/io_relay/multi_turn_encoder/spd_lidar/device。
Core 驱动	BatteryCore、SolarCore、HoistHookCore、IoRelayCore、MultiTurnEncoderCore、SpdLidarCore，由 Interface 按配置创建并驱动，结果写回 Interface 的 latest_*。
ai_safety_common	提供 DeviceStatus、CraneState、AlertMessage 等共享类型（shared_memory_types）。

SDK 架构图

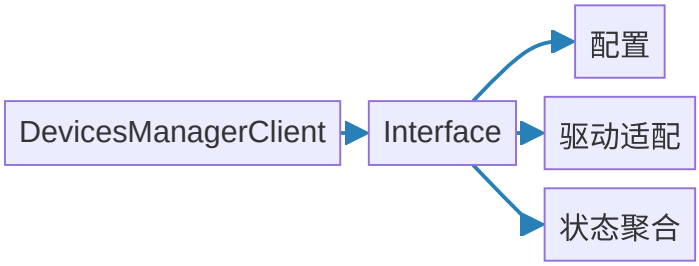
1. 分层架构



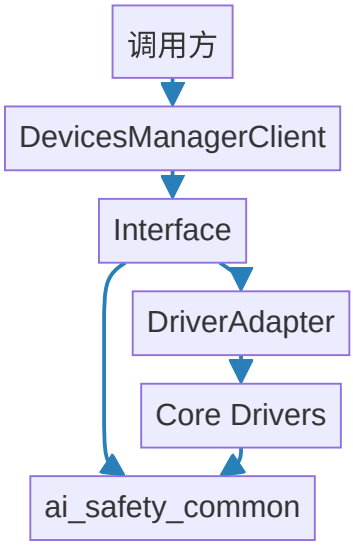
2. 模块依赖（C4）



3. application 内部



4. 依赖方向



5. 架构说明

层级	模块	职责
应用层	main_test / main / demo	调用 DevicesManagerClient，connect Pull/Push 槽，承载业务逻辑
门面层	DevicesManagerClient	统一入口，封装 Interface，暴露 signal-slot 扩展点

业务编排层	Interface	配置加载、驱动创建与调度、状态聚合、轮询线程管理
适配层	DriverAdapter	统一驱动接口，屏蔽各 Core 实现差异
驱动实现层	BatteryCore 等	具体硬件/协议访问（Modbus、TCP、串口等）
基础设施层	SensorFactory、ai_safety_common、config	类型定义、传感器注册、配置文件